CS6382: Algorithm Analysis and Game Theory **Due date:** Oct 30, 23:59

Assignment 1

Student No.: _____ **Student Name:** _____

Instructions

1. Write your solutions by hand, either on paper or on an electronic device.
2. To earn full marks, you only need to complete any **three** of the six questions. Submit a scanned or digital copy of your handwritten solutions via Canvas by **Oct 30, 23:59**.
3. **Late submissions will not be accepted.**

Question 1: Knapsack with two resource constraints

There are $n \geq 1$ items. Item i has value $v_i > 0$ and consumes $a_i, b_i \geq 0$ units of two resources. The respective budgets are $A, B > 0$. Assume $a_i \leq A$ and $b_i \leq B$ for every item, so every singleton is feasible. All values, resource requirements, and budgets are rational and explicitly given.

Find a subset S maximizing

$$v(S) = \sum_{i \in S} v_i$$

subject to

$$\sum_{i \in S} a_i \leq A, \qquad \sum_{i \in S} b_i \leq B.$$

(a) Using x_i to indicate whether item i is selected, write an integer programming formulation and its LP relaxation. Explain why

$$\mathrm{OPT} \leq \mathrm{OPT}_{LP}.$$

(b) Consider $A = B = 1$ and two unit-value items with resource requirements

$$(a_1, b_1) = (1, \delta), \qquad (a_2, b_2) = (\delta, 1), \qquad 0 < \delta < 1.$$

Find the optimal integral solution and the unique optimal LP solution. What happens if every fractional variable is rounded down to zero?

(c) Prove that every extreme point of the LP feasible region has at most two fractional variables. *Hint:* If at least three variables are fractional, find a nonzero perturbation d, supported on those variables, satisfying

$$\sum_i a_i d_i = 0, \qquad \sum_i b_i d_i = 0.$$

(d) Let x^* be an optimal extreme point and define

$$S = \{i : x_i^* = 1\}.$$

Let j be an item of maximum value. Consider the algorithm that returns the better of S and $\{j\}$. Prove feasibility and show that this is a polynomial-time 3-approximation.

You may use the fact that an optimal extreme point of a nonempty bounded rational LP can be found in polynomial time.

Question 2: Weighted vertex cover in a hypergraph

A hypergraph consists of a vertex set V and a collection $\mathcal{E}$ of nonempty subsets of V, called hyperedges. Every hyperedge contains at most r vertices, where $r \geq 2$. Each vertex v has rational cost $c_v > 0$.

A vertex cover is a set $C \subseteq V$ satisfying $C \cap e \neq \varnothing$ for every hyperedge e. The objective is to minimize

$$c(C) = \sum_{v \in C} c_v.$$

All hyperedges are explicitly listed.

(a) Using x_v to indicate whether vertex v is selected, formulate the problem as an integer program and give its LP relaxation. Explain why the LP optimum is a lower bound on OPT.

(b) Let x^* be an optimal LP solution. Consider

$$C = \{v : x_v^* \geq 1/r\}.$$

Prove that C meets every hyperedge.

(c) Prove that this rounding algorithm is a polynomial-time r-approximation.

(d) Consider the hypergraph containing every r-element subset of an n-element vertex set, where $n \geq r$, with unit vertex costs.

Determine:

1. the minimum integral cover size;
2. the LP optimum;
3. the ratio $\mathrm{OPT}/\mathrm{OPT}_{LP}$.

Deduce that the integrality gap approaches r as $n \to \infty$, for fixed r. Does this example, by itself, rule out a polynomial-time approximation factor smaller than r for the problem?

Question 3: Weighted dominating set

Let $G = (V, E)$ be a simple undirected graph of maximum degree Δ. Each vertex v has positive rational cost c_v. Define its closed neighborhood by

$$N[v] = \{v\} \cup \{u : uv \in E\}.$$

A set $D \subseteq V$ is a dominating set if every vertex belongs to $N[v]$ for some $v \in D$. The objective is to minimize $c(D)$.

Consider the following algorithm. Initially $U = V$ and $D = \varnothing$. While $U \neq \varnothing$, choose a vertex v minimizing

$$\frac{c_v}{|N[v] \cap U|}$$

among vertices with positive denominator. Add v to D and remove $N[v]$ from U.

(a) A simpler heuristic chooses the vertex that dominates the largest number of currently undominated vertices, ignoring costs. Construct an instance on three vertices for which this heuristic has an arbitrarily large approximation ratio.

(b) Whenever the stated algorithm selects v, assign a price

$$p_u = \frac{c_v}{|N[v] \cap U|}$$

to each vertex u newly dominated at that step, using U immediately before the selection. Prove that

$$c(D) = \sum_{u \in V} p_u.$$

(c) Fix a vertex v, and list $N[v] = \{u_1, \ldots, u_s\}$ in the order these vertices become dominated, breaking simultaneous ties arbitrarily. Prove that

$$p_{u_j} \leq \frac{c_v}{s - j + 1}.$$

Consequently, show that

$$\sum_{u \in N[v]} p_u \leq H_s c_v, \qquad H_s = \sum_{j=1}^{s} \frac{1}{j}.$$

(d) Use an optimal dominating set to prove that the algorithm is an

$$H_{\Delta+1}\text{-approximation.}$$

Explain why it terminates in polynomial time.

Question 4: Steiner tree through a terminal MST

Let $G = (V, E)$ be a connected undirected graph with positive rational edge costs. A specified set $R \subseteq V$ contains $k \geq 2$ terminals.
A Steiner tree is a tree in G containing all terminals; it may also contain other vertices. The objective is to minimize its total edge cost.
For terminals u, v, let $d(u, v)$ be their shortest-path distance in G. Consider the algorithm:

1. Construct the complete graph on R, giving edge uv cost $d(u, v)$.
2. Compute an MST M of this complete graph.
3. Replace each edge of M by a corresponding shortest path in G.
4. Return a spanning tree A of the union of these paths.

(a) Prove that d satisfies the triangle inequality and that the algorithm can be implemented in polynomial time.
(b) Prove that A is a feasible Steiner tree and

$$c(A) \leq d(M).$$

(c) Let T^* be an optimal Steiner tree. For $k \geq 3$, prove that the terminals have a cyclic ordering $r_1, \ldots, r_k$ satisfying

$$\sum_{i=1}^{k} d(r_i, r_{i+1}) \leq 2c(T^*), \qquad r_{k+1} = r_1.$$

Hint: Double every edge of T^* and consider an Euler tour.
(d) Delete a most expensive edge from the terminal cycle in part (c). Use the resulting path to prove

$$c(A) \leq 2\left(1 - \frac{1}{k}\right) c(T^*).$$

Show separately that the algorithm is exact when $k = 2$.
(e) Show that this guarantee is tight using the following family. There are $k \geq 3$ terminals and one additional vertex s. Every edge from s to a terminal costs 1, and every edge between two terminals costs $2 - \varepsilon$, where

$$0 < \varepsilon < \frac{k-2}{k-1}.$$

Determine OPT, determine the algorithm's output cost, and take the limit as $\varepsilon \downarrow 0$.

Question 5: Rent-or-buy with changing rental prices

You need a machine on each of $T \geq 1$ consecutive days. The horizon T is unknown: at the beginning of a day, you know that service is required that day, but do not know whether another day will follow. Buying the machine permanently costs $B > 0$.

At the beginning of day t, its rental price $r_t > 0$ is revealed. If you have not bought the machine, you may rent it for r_t or buy it for B. Buying occurs before that day's service and avoids that day's rental payment. All future service is then free. All prices are rational. The offline optimum knows T and all prices in advance. Here, c-competitive means ALG $\leq c\,$OPT on every finite input, with no additive constant.

Consider the following *threshold algorithm*. Let R be the total rental payments made so far. Before buying, on each active day:

$$\begin{cases} \text{buy}, & R + r_t \geq B, \\ \text{rent and update } R \leftarrow R + r_t, & R + r_t < B. \end{cases}$$

(a) Prove that

$$\text{OPT} = \min\left\{B, \sum_{t=1}^{T} r_t\right\}.$$

(b) Let $B = 10$ and $(r_1, r_2, r_3, r_4) = (3, 4, 5, 1)$. Determine the threshold algorithm's decisions, its cost, and OPT. A different rule rents until its accumulated rental payments are at least B, and then buys on the next active day. Show that this rule has no finite competitive ratio when rental prices can vary.

(c) Prove that the threshold algorithm maintains $R < B$ before buying. Show that if it never buys, its cost equals OPT; if it buys, then OPT $= B$ and ALG $< 2B$. Deduce that it is 2-competitive.

(d) Fix an integer $M \geq 2$ and restrict attention to inputs with $r_t = B/M$ every day. Determine when the threshold algorithm buys, and show that its exact competitive ratio on this family is $2 - 1/M$. Deduce that its competitive ratio on the full variable-price problem is exactly 2.

(e) Prove that no deterministic online algorithm has competitive ratio smaller than 2 for the full problem.

Hint: First prove a lower bound of $2 - 1/M$ on the family in part (d). Simulate the algorithm on an indefinite constant-price sequence. If its first buying day is τ, stop on day τ and distinguish $\tau \leq M$ from $\tau > M$. Also handle an algorithm that never buys. Finally let M increase.

Question 6: Online matching with capacities

A set V of offline vertices is known in advance. Each vertex $v \in V$ has an integer capacity $b_v \geq 1$, given in binary. Online vertices arrive one at a time in an adversarial order. When an online vertex u arrives, its set of neighbors $N(u) \subseteq V$ is revealed. The algorithm must immediately either assign u to one neighbor with unused capacity or reject u permanently. Assignments are irrevocable, and each online vertex can be assigned at most once. The objective is to maximize the number of assignments.

Consider the following greedy algorithm: whenever an online vertex arrives, assign it to any neighbor with unused capacity, if one exists; otherwise reject it.

For an input sequence I, let $\mathrm{ALG}(I)$ denote the number of assignments made by the algorithm, and let $\mathrm{OPT}(I)$ denote the maximum number achievable by an offline algorithm that knows the entire sequence. An α-competitive algorithm here means one satisfying

$$\mathrm{ALG}(I) \geq \alpha\, \mathrm{OPT}(I)$$

for every finite input sequence I, with no additive term.

(a) Suppose $V = \{a, b\}$, with capacities $b_a = 2$ and $b_b = 1$. Three online vertices arrive in the order u_1, u_2, u_3, with

$$N(u_1) = \{a, b\}, \qquad N(u_2) = \{a\}, \qquad N(u_3) = \{a\}.$$

If greedy prefers a whenever both choices are available, determine its assignments and its objective value. Find an optimal offline assignment.

(b) Describe how to implement greedy using one counter for each offline vertex. Explain why its running time is polynomial in the input length even though the capacities are encoded in binary. Your implementation should not create b_v copies of vertex v.

(c) Fix an input sequence and an optimal offline assignment. Let A be the set of online vertices assigned by greedy, and let O be the set assigned by the optimum. Let

$$S = \{v \in V : \text{greedy uses all } b_v \text{ units of capacity at } v\}.$$

Show that every vertex in $O \setminus A$ is assigned by the optimum to a vertex in S. Deduce that

$$|O \setminus A| \leq \sum_{v \in S} b_v \leq |A|.$$

(d) Use part (c) to prove that greedy is $1/2$-competitive.

(e) Show that no deterministic online algorithm can have competitive ratio strictly greater than $1/2$ in this model.

Hint: Use two offline vertices a, b of capacity 1 each. The first online vertex has neighborhood $\{a, b\}$. Choose the neighborhood of the second online vertex after observing the algorithm's first decision. Your argument should also cover an algorithm that rejects the first vertex.